Министерство цифрового развития

Федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Отчёт

по дисциплине «Тестирование программного обеспечения»
по лабораторной работе № 1
«Разработка автотестов для Web UI с использованием Selenium»

Выполнили:

Студент группы ИП-413
Пряженов В. А.

Работу проверил: Преподаватель
Истомина А. С.

Новосибирск 2026 г.

СОДЕРЖАНИЕ

Введение 3
1 Цель и задачи работы 3
2 Теоретические сведения 4
3 Подготовка окружения 4
4 Структура проекта 5
5 Разработка автотестов 6
6 Фрагменты реализации 7
7 Порядок запуска и демонстрации в WSL 7
8 Результаты запуска и анализ 8
9 Вывод 9

ВВЕДЕНИЕ

В лабораторной работе выполнена разработка набора автоматизированных тестов для веб-интерфейса OpenBMC. Основное внимание уделено проверке авторизации, так как вход в Web UI является обязательной точкой доступа к функциям управления BMC. Дополнительно подготовлены два тестовых сценария по функциям веб-интерфейса, которые соответствуют проверкам из предыдущей лабораторной работы: управление питанием и просмотр событий.

Работа выполнена в среде WSL в каталоге ~/tpo/lab4_selenium_openbmc. Для запуска целевой системы использован образ OpenBMC в QEMU, доступный по HTTPS на локальном порту 2443. Для реализации автотестов применены Python, pytest и Selenium WebDriver.

1 ЦЕЛЬ И ЗАДАЧИ РАБОТЫ

Цель работы - изучить принципы разработки автотестов для Web UI с использованием Selenium и применить их для проверки авторизации и отдельных функций OpenBMC.

- изучить назначение Selenium WebDriver и pytest при автоматизации пользовательских сценариев;
- подготовить проект автотестов в WSL и описать зависимости проекта;
- разработать тест успешной авторизации в OpenBMC Web UI;
- разработать тест авторизации с некорректными учетными данными;
- разработать тест обработки нескольких неудачных попыток входа;
- добавить два Web UI теста по сценариям предыдущей лабораторной работы;
- запустить тесты, проанализировать результат выполнения и подготовить проект к публикации на GitHub.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Selenium WebDriver

Selenium WebDriver - инструмент для автоматизации действий пользователя в браузере. Тестовый код управляет браузером через драйвер: открывает страницы, вводит данные, нажимает кнопки, ожидает появления элементов и проверяет итоговое состояние интерфейса. Такой подход позволяет проверять систему ближе к реальному поведению пользователя, чем модульные тесты отдельных функций.

2.2 pytest

pytest используется как фреймворк запуска тестов. Он собирает функции с именами test_*, подготавливает фикстуры, передает в тесты параметры окружения, формирует отчет в консоли и позволяет помечать тесты маркерами. В проекте используются маркеры auth и webui, чтобы отделять проверки авторизации от остальных сценариев веб-интерфейса.

2.3 Особенности OpenBMC Web UI

OpenBMC предоставляет веб-интерфейс для управления BMC и серверной платформой. В зависимости от версии образа и сборки интерфейс может отличаться: часть образов открывает полноценную страницу входа, а часть отвечает через HTTP Basic Auth. Поэтому вспомогательный класс LoginPage в тестовом проекте учитывает оба варианта: форму логина и базовую HTTP-авторизацию.

Для стабильности автотестов используются явные ожидания. Тест не должен полагаться только на фиксированную задержку, потому что время загрузки страницы зависит от состояния QEMU, производительности компьютера и текущей загрузки системы.

3 ПОДГОТОВКА ОКРУЖЕНИЯ

Перед разработкой автотестов был подготовлен каталог лабораторной работы в WSL. В каталоге создано виртуальное окружение Python, добавлены зависимости и создана структура tests. Целевая система OpenBMC

запускалась в QEMU из каталога ~/tpo, где расположен файл образа obmc-phosphor-image-romulus.static.mtd.

Компонент	Назначение
WSL	Среда выполнения лабораторной работы и хранения проекта в каталоге ~/tpo.
QEMU	Эмулятор платы Romulus BMC для запуска образа OpenBMC.
OpenBMC	Тестируемая система с Web UI, доступная по адресу https://127.0.0.1:2443.
Python	Язык реализации автотестов и вспомогательных классов.
pytest	Фреймворк запуска тестов, фикстур, маркеров и отчетов.
Selenium	Библиотека управления браузером через WebDriver.
Google Chrome	Браузер, в котором выполнялись Selenium-тесты.

Для передачи параметров тестам используются переменные окружения. Такой вариант удобнее, чем хранение адреса и пароля прямо в тестовом коде: при изменении стенда достаточно изменить переменные, а не исходные файлы.

4 СТРУКТУРА ПРОЕКТА

Проект организован как небольшой pytest-пакет. Фикстуры и вспомогательные классы вынесены отдельно, чтобы тестовые функции оставались читаемыми и описывали именно проверяемый сценарий.

Файл	Содержание
requirements.txt	Список зависимостей: pytest и selenium.
pytest.ini	Настройки pytest: режим подробного вывода, каталог tests, маркеры auth и webui.
tests/conftest.py	Фикстуры browser, base_url, credentials и driver; настройка Chrome; скриншоты при ошибках.
tests/pages.py	Вспомогательные классы LoginPage и OpenBmcUi, функции ожидания и поиска элементов.
tests/test_openbmc_auth_ui.py	Три теста авторизации: успешный вход, неверные данные, повторные неверные попытки.

tests/test_openbmc_webui_lab3.py	Два теста Web UI по предыдущей лабораторной работе: питание и события.
reports/pytest_openbmc_run.txt	Сохраненный вывод pytest после выполнения тестов.

5 РАЗРАБОТКА АВТОТЕСТОВ

Для лабораторной работы реализовано пять тестовых сценариев. Три сценария проверяют авторизацию, а два дополнительных сценария относятся к функциям OpenBMC Web UI, выбранным из задания предыдущей лабораторной работы. В текущем образе OpenBMC часть страниц Web UI недоступна, поэтому для таких проверок предусмотрено корректное пропускание теста через pytest.skip с пояснением причины.

ID	Проверка	Шаги	Ожидаемый результат	Факт
AUTH-01	Успешная авторизация	Открыть Web UI, ввести root и корректный пароль, выполнить вход.	Пользователь получает доступ к защищенному интерфейсу.	Passed
AUTH-02	Неверные учетные данные	Открыть Web UI, ввести неправильный логин и пароль.	Система отклоняет вход и возвращает признак ошибки авторизации.	Passed
AUTH-03	Повторные неверные попытки	Три раза выполнить вход с неправильным паролем.	Система стабильно обрабатывает ошибки и не предоставляет доступ.	Passed
WEB-01	Доступность управления питанием	После входа найти раздел Power или Operations и проверить текст страницы.	Страница питания доступна и содержит признаки действий On, Off, Restart или Reboot.	Skipped
WEB-02	Доступность журнала событий	После входа найти раздел Logs, Event Log или Events и проверить текст страницы.	Страница логов доступна и содержит сведения о событиях.	Skipped

Пропуск WEB-01 и WEB-02 не означает ошибку разработки тестов. Он фиксирует фактическое состояние стенда: в текущей сборке OpenBMC страницы питания и событий не отображаются в доступной навигации Web UI. Если запустить тесты на сборке OpenBMC с полноценным веб-интерфейсом, сценарии выполнят поиск нужных страниц и проведут проверки содержимого.

6 ФРАГМЕНТЫ РЕАЛИЗАЦИИ

6.1 Зависимости проекта

```
pytest==8.3.5
selenium==4.25.0
```

6.2 Настройки pytest

```
[pytest]
addopts = -ra -v
testpaths = tests
markers =
    auth: OpenBMC Web UI authentication tests
    webui: OpenBMC Web UI feature tests
```

6.3 Проверка успешной авторизации

```
@pytest.mark.auth
def test_successful_login(driver, base_url, credentials):
    login_page = LoginPage(driver, base_url).open()
    login_page.login(credentials['username'], credentials['password'])
    OpenBmcUi(driver).assert_logged_in()
```

6.4 Проверка некорректных учетных данных

```
@pytest.mark.auth
def test_invalid_login_shows_error(driver, base_url):
    login_page = LoginPage(driver, base_url).open()
    login_page.login('wrong_user', 'wrong_password')
    message = login_page.error_text().lower()
    assert any(word in message for word in ['invalid', 'error', 'failed', 'невер',
'ошиб'])
```

6.5 Проверка страницы питания

```
@pytest.mark.webui
def test_power_action_is_available_in_web_ui(driver, logged_in):
    ui = logged_in
    opened = ui.open_page_by_text(['operations', 'power', 'server power',
'управление', 'питание
    '])
    if not opened:
        pytest.skip('Power or operations page was not found in the current OpenBMC Web
UI build'
    )
    page_text = visible_text(driver)
    assert any(word in page_text for word in ['power', 'on', 'off', 'restart',
'reboot', 'пит'])
```

7 ПОРЯДОК ЗАПУСКА И ДЕМОНСТРАЦИИ В WSL

7.1 Запуск OpenBMC в первом окне WSL

Для демонстрации лабораторной работы необходимо открыть первое окно WSL, перейти в каталог ~/tpo и запустить QEMU. Это окно остается занятым процессом эмулятора до завершения проверки.

```
cd ~/tpo
qemu-system-arm -m 256 -M romulus-bmc -nographic \
  -drive file=obmc-phosphor-image-romulus.static.mtd,format=raw,if=mtd \
  -net nic \
  -net user,hostfwd=:0.0.0.0:2222-:22,hostfwd=:0.0.0.0:2443-
:443,hostfwd=udp:0.0.0.0:2623-:623,h
    ostname=qemu
```

7.2 Запуск автотестов во втором окне WSL

После загрузки OpenBMC открывается второе окно WSL. В нем выполняется переход в каталог проекта, активация виртуального окружения, установка зависимостей и запуск pytest.

```
cd ~/tpo/lab4_selenium_openbmc
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
export OPENBMC_BASE_URL='https://127.0.0.1:2443'
export OPENBMC_USERNAME='root'
export OPENBMC_PASSWORD='0penBmc'
pytest -v
```

7.3 Что показывать при защите

- первое окно WSL с запущенным QEMU и проброшенным портом 2443;
- каталог ~/tpo/lab4_selenium_openbmc со структурой tests, requirements.txt и pytest.ini;
- команду pytest -v и итоговую строку выполнения тестов;
- файл reports/pytest_openbmc_run.txt с сохраненным результатом прогона;
- исходные файлы tests/test_openbmc_auth_ui.py и tests/test_openbmc_webui_lab3.py.

8 РЕЗУЛЬТАТЫ ЗАПУСКА И АНАЛИЗ

Автотесты были запущены против OpenBMC, работающего в QEMU. Перед запуском тестов был проверен ответ веб-сервера по адресу https://127.0.0.1:2443. В текущем образе сервер возвращал HTTP Basic Auth, что было учтено в реализации LoginPage.

```
tests/test_openbmc_auth_ui.py::test_successful_login PASSED
tests/test_openbmc_auth_ui.py::test_invalid_login_shows_error PASSED
tests/test_openbmc_auth_ui.py::test_repeated_invalid_login_is_handled PASSED
tests/test_openbmc_webui_lab3.py::test_power_action_is_available_in_web_ui SKIPPED
tests/test_openbmc_webui_lab3.py::test_event_logs_are_available_after_opening_logs_pag
e SKIPPED
3 passed, 2 skipped in 325.03s (0:05:25)
```

Результат показывает, что обязательные проверки авторизации выполнены успешно. Тесты по страницам питания и журналов корректно отработали условие отсутствия нужных страниц в текущем Web UI и были пропущены с диагностикой. Такой результат является приемлемым для используемого стенда, поскольку тестовая программа не завершается ошибкой, а явно сообщает о причине пропуска.

9 ВЫВОД

В результате выполнения лабораторной работы был подготовлен проект автотестов для OpenBMC Web UI. Реализованы тесты успешной авторизации, авторизации с неверными данными, обработки повторных неверных попыток, а также два дополнительных сценария по функциям Web UI. Проект содержит настройки pytest, зависимости, фикстуры Selenium WebDriver и вспомогательные классы для работы со страницами.

Пробный запуск подтвердил работоспособность тестов авторизации: три теста прошли успешно. Два теста по страницам питания и событий были пропущены из-за особенностей текущей сборки OpenBMC, что отражено в отчете pytest. Подготовленный проект можно использовать как основу для дальнейшего расширения набора Web UI автотестов.